

Building a Local AI Morning Briefing

An AI Engineering Case Study — Step by Step

Carlos Lucero · August 4, 2026

Stack: Hermes Agent (Nous Research) · Ollama + hermes3 8B · RTX 5060 8GB · Python · WhatsApp

What this document covers: In one working session we repaired a broken scheduled AI job, then designed, built, broke, debugged, and re-architected a daily automated briefing that runs entirely on a local 8-billion-parameter model. Every concept along the way is a core AI engineering idea: model pinning, knowledge cutoffs, retrieval by pre-fetching, context budgets, agentic tool calling, task decomposition, prompt design for small models, output guardrails, and observability through logs. The final system delivers a personalized greeting, a rotating learning topic, and sourced AI news to WhatsApp every morning at 5:30 — at zero API cost.

The System at a Glance

The finished pipeline has five stages. Understanding who does what — and, critically, which stages are deterministic code versus generative AI — is the single most important design idea in this project.

```
5:30 AM  Hermes cron scheduler fires the job
|
v
fetch_ai_news.py (deterministic Python)
  1. Pull 8 RSS feeds (TechCrunch, Verge, Guardian, SCMP, ...)
  2. Filter to last 26h, dedupe, sort into 5 focus buckets
  3. Pick today's learning topic from a 60-topic curriculum
  |
  v
Ollama / hermes3 (generative, 2 small focused calls)
  4a. Write the morning greeting      (temperature 0.9)
  4b. Explain the learning topic      (temperature 0.4)
  |
  v
Script assembles the final message (deterministic again)
  |
  v
Hermes gateway delivers stdout verbatim -> WhatsApp
```

The model never touches the news headlines. The script never writes prose. Each half does what it is reliable at. We did not start with this design — we arrived at it after the first architecture failed, and that failure is the most instructive part of the story.

Part 1 — The Broken Cron Job: Config Drift and Model Pinning

The symptom

Every morning at 7:00, a WhatsApp message reported that the existing NYT digest job had failed:

```
Cron 'nyt-digest' failed: Skipped to prevent unintended spend:
global inference config drifted since this job was created
(provider 'anthropic' -> 'custom';
 model 'claude-haiku-4-5' -> 'hermes3'), and this job is unpinned.
No inference call was made.
```

The concepts

- **Provider and model.** An inference provider is the service that runs the model (Anthropic's API, or your local Ollama server). Any app that calls an LLM stores both: who to call, and which model to ask for.
- **Config snapshot.** When the job was created, Hermes recorded the provider and model active at that moment (anthropic / claude-haiku). That frozen record is a snapshot.
- **Config drift.** Later, the global config was switched to local Ollama (provider 'custom', model 'hermes3'). Now the snapshot and the live config disagree — that difference is drift.
- **The drift guard.** Rather than silently run the job on a different provider than the one it was written for — possibly billing an account the owner abandoned, or producing different-quality output — Hermes refused to run and reported why. Failing loudly instead of guessing is good engineering. Your morning error messages were the guard working as designed.

- **Pinning.** The fix is to make the job's intent explicit: pin the provider and model onto the job itself so there is nothing to guess about.

The fix (one command)

```
hermes cron edit c4254649cef5 --model hermes3 --provider custom
```

Lesson: scheduled AI jobs should always pin their model. Model choice is part of the job's contract — output quality, cost, and context limits all change when the model does.

Part 2 — Why a Local Model Cannot Read the News

The goal: a 5:30 AM briefing with fresh AI headlines, written by hermes3 running on the local GPU. The immediate problem is fundamental to how LLMs work.

- **Knowledge cutoff.** A model's weights are frozen at training time. hermes3 knows nothing that happened after its training data was collected — and it has no network access. Asked for today's news, it can only hallucinate plausible-sounding stories.
- **The pre-fetch pattern.** The universal solution: fetch fresh data with ordinary code, then hand it to the model as part of the prompt. This is the same principle behind RAG (Retrieval-Augmented Generation) — the retrieval here is simply 'download today's RSS feeds' instead of 'search a vector database'.
- **Context budget.** The GPU has 8 GB of VRAM, which caps hermes3 at a 16,384-token context window. Everything — instructions, news data, and the answer — must fit. This is why the fetch script caps stories per category, trims descriptions, and drops anything older than 26 hours. Counting tokens before you spend them is a daily AI engineering activity.

Part 3 — The Data Layer: Fetching and Structuring the News

Reading RSS with nothing but the standard library

RSS is a decades-old XML format most news sites still publish; it needs no API keys and no scraping. Eight feeds were chosen to cover the five focus areas (data centers, China, regulation, jobs, environment). Two formats exist in the wild — RSS 2.0 and Atom — so the parser handles both:

```
def parse_feed(raw):
    """Yield (title, desc, published) from RSS 2.0 or Atom."""
    root = ET.fromstring(raw)
    ns = {"atom": "http://www.w3.org/2005/Atom"}
    items = root.findall("./item")
    if items: # RSS 2.0
        for it in items:
            title = strip_tags(it.findtext("title"))
            desc = strip_tags(it.findtext("description"))
            when = parse_date(it.findtext("pubDate"))
            if title:
                yield title, desc, when
        return
    for it in root.findall("./atom:entry", ns): # Atom
        title = strip_tags(it.findtext("atom:title", namespaces=ns))
        ...
```

Fault tolerance: one dead feed must not kill the briefing

```

for source, url in FEEDS:
    try:
        for title, desc, when in parse_feed(fetch(url)):
            ...
    except Exception as exc:
        errors.append(f"{source}: {type(exc).__name__}")

```

Every external dependency is a thing that will eventually fail at 5:30 AM. Wrapping each feed independently means a Guardian outage costs you Guardian stories, not your whole briefing.

Dedup and keyword bucketing

Multiple outlets cover the same story, so titles are normalized (lowercased, stripped to alphanumerics) and used as a seen-set key. Then simple keyword matching sorts each story into the first bucket it hits — priority order matters, so a story about Chinese data centers files under DATA CENTERS rather than CHINA:

```

def bucket_for(title, desc):
    text = f"{title} {desc}".lower()
    for name, keywords in BUCKETS:      # priority order
        if any(k in text for k in keywords):
            return name
    return OTHER_BUCKET

```

Note what this is not: it is not an LLM classifier. Keyword matching is crude but free, instant, and 100% predictable — and for five broad topics it is plenty. Reaching for the model when a list of strings will do is one of the most common beginner mistakes in AI engineering.

The rotating curriculum: state across runs

The learning topic advances daily through a 60-topic list ordered like a curriculum (tokenization -> embeddings -> attention -> RAG -> fine-tuning -> agents -> LLMOps). Cron jobs are stateless — each run is a fresh process — so the current position lives in a tiny JSON state file:

```

def pick_topic():
    idx = 0
    try:
        idx = int(json.loads(STATE_FILE.read_text())["index"])
    except (OSError, ValueError, KeyError, json.JSONDecodeError):
        pass      # missing/corrupt state -> start at 0
    topic = TOPICS[idx % len(TOPICS)]
    STATE_FILE.write_text(json.dumps({"index": (idx + 1) % len(TOPICS)}))
    return topic

```

The broad except is deliberate: if the state file is missing or mangled, the job must still run. Defaulting to a sane value beats crashing before dawn.

Part 4 — First Attempt: The Agent Path, and Why It Failed

The original design

Version 1 followed the pattern of the existing NYT job: the script writes the news to a file, and the job prompt tells the model to read that file with its 'file' tool, then write the whole briefing — greeting, lesson, and news — in one large generation.

```
# Version 1 (FAILED):
#   prompt: "Read the file ../ai_morning_latest.md ...
#           write greeting + topic + news, under 450 words, ..."
#   job toolsets: [file]   model: hermes3 via agent loop
```

The failure

```
API call #1: model=hermes3 in=1904 out=185 latency=9.1s
WARNING Empty response (no content or reasoning) - retry 1/3
API call #2: model=hermes3 in=1904 out=205
WARNING Empty response (no content or reasoning) - retry 2/3
API call #3: model=hermes3 in=1904 out=386
WARNING Empty response (no content or reasoning) - retry 3/3
Turn ended: reason=empty_response_exhausted tool_turns=0
```

Read the numbers carefully — this is what real LLM debugging looks like. The model received 1,904 input tokens and generated 185-386 output tokens on every attempt, yet the agent framework saw no usable content and no tool calls. The model was producing something, but not something the agent loop could use.

Isolating the fault

Debugging rule: reproduce with the smallest possible test. First, plain chat against Ollama's OpenAI-compatible endpoint — worked. Next, the same endpoint with a tool definition attached:

```
POST http://localhost:11434/v1/chat/completions
{ "model": "hermes3",
  "messages": [{ "role": "user",
                  "content": "Read the file C:/test.md and summarize it." }],
  "tools": [{ "type": "function", "function": {
               "name": "read_file", ... } } ] }

-> content: '' (empty)
-> tool_calls: read_file({"path": "C:\\test.md"})
-> finish_reason: tool_calls
```

Tool calling itself worked in isolation. But inside the full agent loop — with the cron system prompt, delivery instructions, and a multi-part task — the 8B model kept producing malformed turns that parsed as neither text nor a valid tool call. After a second full-quality failure (the model ignored half the instructions and opened with a phrase from its own banned list), the verdict was clear.

The real lesson

Small models are far more capable as writers than as agents. An agentic step (decide to call a tool, format the call correctly, consume the result, then write) multiplies failure points, and an 8B model fails one of them often enough to be unreliable. A frontier model shrugs this task off; a local 8B model needs the task reshaped. Matching architecture to model capability — instead of assuming capability — is a defining skill of production AI engineering.

Part 5 — The Redesign: Task Decomposition

The principle

Split the job by reliability requirements. Facts (headlines, sources, structure) must be exact -> deterministic code. Personality (greeting, explainer) must be fresh and human -> the model, but with one small job per call and no tools anywhere.

Aspect	V1 — agent mode (failed)	V2 — no-agent mode (works)
Who assembles the message	The model, in one big generation	Python, deterministically
News headlines	Model paraphrases (mangled them)	Copied exactly from feeds
Model calls	1 large agentic turn + file tool	2 small text-only calls
Tool calling required	Yes — the fragile step	None
Failure mode	Empty responses, slop, fabrication	Worst case: fallback text
Delivery	Agent's final response	Script stdout, verbatim

Calling Ollama directly

With no agent in the loop, the script talks to Ollama's native REST API itself — an LLM call is, in the end, just an HTTP request:

```
def ollama(prompt, temperature, num_predict):
    body = json.dumps({
        "model": "hermes3",
        "prompt": prompt,
        "stream": False,
        "options": {"temperature": temperature,
                    "num_predict": num_predict},
    }).encode("utf-8")
    req = urllib.request.Request(OLLAMA_URL, data=body,
        headers={"Content-Type": "application/json"})
    with urllib.request.urlopen(req, timeout=300) as resp:
        return json.loads(resp.read())["response"].strip()
```

Temperature: one knob, two settings

Temperature controls sampling randomness. The two creative calls want different behavior, so they get different values — tuning temperature per task, not per app, is the professional habit:

Call	Temperature	Why
Morning greeting	0.9 (high)	Variety and personality; a different greeting every day
Topic explainer	0.4 (low)	Factual accuracy matters more than flair

Prompting a small model: the greeting

Three techniques doing the heavy lifting: (1) one focused job per prompt; (2) a style example to anchor the voice — showing beats telling; (3) a daily rotating 'angle' so a high temperature has fresh raw material and the model does not converge on one stock greeting:

```
GREETING_ANGLES = [
    "the Texas heat and getting after it before the sun does",
    "learning AI at 47 while half the industry burns out at 28",
    "gray in the beard, CUDA cores in the office",
    ...
]

random.seed(today.toordinal())      # same angle all day,
angle = random.choice(GREETING_ANGLES) # new angle tomorrow
```

Guardrails: trust, but verify the output

The first test run opened with 'Rise and grind' — a phrase the prompt explicitly banned. Models, especially small ones, violate 'don't say X' instructions constantly. The engineering answer is to validate output in code and retry, with a hand-written fallback so the pipeline can never emit slop or nothing:

```
BANNED = ["rise and grind", "grind", "crush it", "unleash",
          "hustle", "conquer", "seize the day", "level up", ...]

def make_greeting(today):
    ...
    for _ in range(3):
        text = clean_model_text(
            ollama(prompt, temperature=0.9, num_predict=120))
        if text and not has_banned(text) and len(text) < 500:
            return text # passed the filter
    return "Morning, Carlos. The coffee's hot, the GPUs are"
        " humming, and you're still the guy who shows up..."
```

This generate -> validate -> retry -> fallback loop is the same shape used in production systems for JSON schema validation, safety filtering, and factuality checks. Never let raw model output reach the user unchecked.

Deterministic assembly and delivery

The script concatenates the pieces and prints. In Hermes's no-agent cron mode, stdout is delivered verbatim to WhatsApp — so the printed string is the message, character for character. WhatsApp renders **asterisked text** as bold, giving clean section headers for free:

```
parts = [
    f"*{today:%A, %B %d} - Morning Briefing*", "",
    greeting, "",
    f"*TODAY'S AI TOPIC: {topic}*", explainer, "",
    f"*AI NEWS - {total} stories*", "", news,
]
print("\n".join(parts)) # stdout IS the WhatsApp message

# Switching the job over:
hermes cron edit d47b8cf1e028 --no-agent
```

Part 6 — Verification: Logs Are Your Eyes

Nothing was declared working until the logs proved it. The debugging sequence that cracked V1 — and confirmed V2 — was entirely log-driven:

1. Run the smallest reproducible test (curl the model API directly; 'Say hello in five words').
2. Escalate one variable at a time (add a tool definition; observe tool_calls appear).
3. Read the run history (hermes cron runs) and the agent log: token counts in/out, latency, turn-end reasons.
4. After the redesign, confirm end to end: 'Job morning-briefing completed successfully ... delivered to whatsapp:12147...'

Token counts told the V1 story (output was being generated but discarded); the delivery line told the V2 story. Observability — logging model calls, tokens, latencies, and outcomes — is what separates an AI system you can fix from one you can only restart.

Ten Lessons for an AI Engineer

1. Pin your models. A scheduled AI job should name its provider and model explicitly; config drift on an unpinned job is a silent behavior change.
2. LLMs cannot know anything after their training cutoff. Fresh data must be fetched by code and injected into the prompt (the retrieval in RAG).
3. Respect the context budget. VRAM caps context; context caps how much data you can inject. Trim, cap, and filter before the model ever sees it.
4. Use code for facts, models for prose. Anything that must be exact (headlines, sources, structure, numbers) should never pass through a generative rewrite.
5. Small models are writers, not agents. Every agentic step (tool choice, call formatting, result handling) is a failure point; an 8B model needs those steps removed, not retried harder.
6. Decompose tasks. Two small single-purpose prompts beat one large multi-part prompt — dramatically so below ~30B parameters.
7. Tune temperature per call: high for creative variety, low for factual explanation.
8. Never trust 'don't say X'. Validate output in code, retry on violation, and keep a deterministic fallback so the pipeline degrades gracefully instead of failing.
9. Design for partial failure. Eight feeds means eight chances to fail at 5:30 AM; isolate each one, and let the briefing ship without the piece that broke.
10. Read the logs before forming a theory. Token counts, finish reasons, and turn-end states told us exactly what was wrong — no guessing required.

Appendix — Command Cheat Sheet

```
# All commands run from PowerShell; hermes.exe lives in
# C:\Users\mrcar\AppData\Local\hermes\hermes-agent\venv\Scripts\

hermes cron list           # list scheduled jobs
hermes cron runs <job_id> # execution history + errors
hermes cron edit <job_id> --model hermes3 --provider custom
hermes cron edit <job_id> --no-agent      # stdout-verbatim mode
hermes cron run <job_id>    # trigger a test run now
hermes gateway restart     # reload .env / platform config

# Quick Ollama health checks
GET http://localhost:11434/api/tags      # installed models
POST http://localhost:11434/api/generate # native one-shot
POST http://localhost:11434/v1/chat/completions # OpenAI-compat
```

Key files: scripts/fetch_ai_news.py (the whole pipeline), cron/jobs.json (job definitions), data/ai_learning_state.json (curriculum position), logs/agent.log and logs/gateway.log (observability), all under C:\Users\mrcar\AppData\Local\hermes\.